

Iniziato lunedì, 26 gennaio 2026, 14:05

Stato Completato

Terminato lunedì, 26 gennaio 2026, 15:01

Tempo impiegato 56 min. 14 secondi

Domanda 1

Completo

Punteggio max.: 2,00

Le applicazioni web server-side realizzate con Spring Web sono basate sul pattern Model-View-Controller. Quale delle seguenti corrispondenze con gli elementi costitutivi di un'applicazione Spring Web è corretta?

- ☐ a. Model = @Entity / View = body della HTTP Response / Controller = @Service, @JpaRepository
- ☐ b. Model = database / View = @RestController / Controller = @Service, @JpaRepository
- ☐ c. Model = @JpaRepository / View = @RestController / Controller = @Service
- ☒ d. Model = @Service, @Entity / View = body della HTTP Response / Controller = @RestController

Domanda 2

Completo

Punteggio max.: 2,00

Quale delle seguenti affermazioni sugli **array** in Javascript è **FALSA**?

- ☐ a. La lunghezza di un array (length) conteggia anche eventuali elementi vuoti presenti nell'array
- ☒ b. La lunghezza dell'array deve essere dichiarata al momento della sua creazione
- ☐ c. Gli array possono contenere dati di tipi eterogenei
- ☐ d. Gli array sono oggetti, dunque hanno proprietà e metodi

Domanda 3

Completo

Punteggio max.: 2,00

Qual è il ruolo dei @Service in un'applicazione web server-side realizzata con Spring/SpringBoot?

- ☐ a. Definire l'API dell'applicazione, ossia a quali richieste è in grado di rispondere
- ☐ b. Offrire accesso al modello dati dell'applicazione, descrivendo le entità rilevanti e le loro relazioni
- ☒ c. Implementare una porzione internamente coesa della business logic dell'applicazione
- ☐ d. Implementare l'Object Relational Mapping, ossia definire la corrispondenza fra entità del DB e oggetti Java

Domanda **4**

Completo

Punteggio max.: 2,00

Una **differenza** fra una **richiesta HTTP** con metodo **GET** e una con metodo **POST** è che...

- ☐ a. una richiesta GET può contenere, nella URL, una *query string*, mentre una richiesta POST no
- ☐ b. una richiesta POST può partire solo da un form, mentre una richiesta GET può partire da un link o dal codice Javascript
- ☐ c. una richiesta GET può utilizzare i cookie, una richiesta POST no
- ☒ d. nelle richieste POST il *body* è usato per i dati da inviare, nelle richieste GET è assente o ignorato

Domanda **5**

Completo

Punteggio max.: 4,67

Illustrare le macrocategorie in cui è possibile suddividere le funzionalità offerte dalla DOM API del browser

La DOM API e' una interfaccia implementata dal browser che grazie a javascript ci permette di interagire con il documento HTML tramite apposito oggetto

Le macrocategorie sono:

- Navigare nell'albero rappresentante il documento

possiamo grazie a questa funzionalita' navigare nella struttura dell'HTML comprendendone la struttura stessa

- Modificare albero documento

con questa si puo' modificare direttamente il DOM riuscendo quindi a smontare/rimontare dal documento degli elementi togliendoli proprio dalla memoria del browser (in modo differente al display: none), a modificarli proprio come cambiare testo o HTML interno al tag o addirittura il loro stile CSS

per modificarli li si seleziona con metodi come getElementById oppure querySelector che sfruttano i selector CSS e poi li si puo' modificare ad esempio assegnando all'attributo innerHTML una stringa rappresentante l'HTML di rimpiazzo dentro l'elemento

- Aggiungere i listeners

ultima ma non per importanza e' il poter aggiungere listeners che agiscono sugli eventi esogeni (dall'utente) che possono avvenire

in qualunque momento e vanno gestiti in modo asincrono, ad esempio: l'onClick

Domanda **6**

Completo

Punteggio max.: 4,66

Delineare le principali funzionalità di un Web Server

Un Web Server principalmente ha 3 compiti:

- Rimanere in ascolto per request
- Elaborare la richiesta
- Inviare la risposta al Client che ha mandato la request

Ma puo' fare altre funzionalita' importanti come:

- Caching

essenziale, soprattutto in server che devono elaborare tantissime richieste al giorno per alleggerire in modo significativo il proprio carico dando all'utente le risposte alle stesse richieste gia' elaborate in precedenza

- Strozzamento della banda

anche se sembra paradossale anche questo serve a molto per dare una esperienza decente a tutto gli utenti che lo utilizzano, dato che se si da troppa banda ad un utente che sta scaricando un video ad esempio gli altri avranno una esperienza a "scatti"

- Virtual Hosting

se sullo stesso web server girano piu' siti web, come in quelli della cloudflare deve poter smistare le richieste che arrivano allo stesso indirizzo IP ma che vogliono siti diversi, lo si capisce tramite hostname (succede cosi' anche per il sito dell'UNITO)

- Autenticazione e Autorizzazione

per la sicurezza e' necessario poter autenticare gli utenti dandogli soltanto i permessi necessari per le operazioni che vogliono fare

- Rewrite Engine

in siti web complicati alcuni URL grezzi potrebbero essere molto lunghi e molto complicati quindi si preferisce fare vedere all'utente una versione clean dell'URL

Domanda **7**

Completo

Punteggio max.: 2,00

Qual è la **caratteristica fondamentale** di un **ipertesto**, senza la quale non è possibile definirlo tale?

- ☐ a. Un ipertesto è necessariamente scritto in linguaggio HTML e trasmesso attraverso una connessione HTTP.
- ☐ b. Un ipertesto prevede di corredare il testo con informazioni su come deve essere reso graficamente (tipo di font, dimensioni del carattere, colore, grassetto o corsivo, ecc.)
- ☒ c. Un ipertesto non prevede un'unica sequenza di lettura; in alcuni punti il lettore può saltare a un punto diverso del testo medesimo, o ad un'altro documento, scegliendo così un proprio percorso di lettura fra quelli disponibili.
- ☐ d. Un ipertesto non prevede soltanto testo propriamente detto, ma può anche contenere altri tipi di risorse, quali immagini, video, audio, o anche codice.

Domanda **8**

Completo

Punteggio max.: 2,00

Si consideri il seguente frammento di HTML:

```
<html><head>...</head>
<body>
  <section>
    <button id="ok">Conferma</button>
    <button id="cancel">Annulla</button>
    <button id="save">Salva e continua dopo</button>
  </section>
</body>
</html>
```

L'utente clicca sul BUTTON con id="ok" (per brevità BUTTON#ok). Si genera dunque un evento di tipo **click** con target BUTTON#ok. Il meccanismo del **bubbling** fa sì che...

- ☒ a. L'evento venga ricevuto anche dall'elemento BODY, *dopo* essere arrivato all'elemento BUTTON#ok
- ☐ b. L'evento venga ricevuto anche dagli elementi BUTTON#cancel e BUTTON#save, *dopo* essere arrivato BUTTON#ok
- ☐ c. L'evento venga ricevuto anche dall'elemento BODY, *prima* di arrivare all'elemento BUTTON#ok
- ☐ d. L'evento venga ricevuto anche dagli elementi BUTTON#cancel e BUTTON#save, *prima* di arrivare all'elemento BUTTON#ok

Domanda **9**

Completo

Punteggio max.: 4,67

Descrivere il ciclo di vita delle @Entity nella Java Persistence API di Spring

Le @Entity rappresentano le tabelle nel database ma le loro modifiche vengono rispecchiate nel DB concreto quando si trovano nello stato managed e non quando sono detached

Secondo la Java Persistence API le @Entity si possono trovare in 2 stati principalmente: managed, detached.

Si entra nello stato managed quando si fanno operazioni definite dalla API come il .save() in cui si prende l'istanza delle entity e se non già presente nel DB viene aggiunta

Lo stato managed è delicato infatti le operazioni devono essere atomiche e non interrotte da un'altra operazione nello stesso momento. Per questo motivo le operazioni di basso livello sono atomiche ed esiste il @Transactional quando si vogliono fare più operazioni insieme e più complesse nella business logic

Nello stato detached invece esattamente l'opposto, ogni modifica fatta all'entità saranno "silenziose" fino a quando non si entra in managed tramite chiamata apposita.

Si entra in questo stato quando è appena finita un'operazione atomica o un metodo @Transactional o è stata appena creata l'entità

Domanda **10**

Completo

Punteggio max.: 2,00

Un componente React è definito come una funzione, detta **funzione di rendering**.

Quale dei seguenti accadimenti implica **sempre e comunque** che il framework invochi di tale funzione, provocando il re-rendering del componente?

- ☐ a. L'interazione da parte dell'utente con la view del componente
- ☐ b. La risoluzione di una Promise legata ad una richiesta HTTP formulata con **fetch**
- ☐ c. L'esecuzione di un effetto definito nel componente tramite useEffect
- ☒ d. La modifica del valore di una variabile di stato definita con useState

Domanda **11**

Completo

Punteggio max.: 2,00

Abbiamo detto che Javascript è un linguaggio **object-oriented** **basato su prototipi**. Questo significa che...

- ☐ a. ...che gli oggetti vengono creati a partire da un costruttore, ma il motore di esecuzione non mantiene alcun legame fra un oggetto e il costruttore con cui è stato creato
- ☐ b. ...che gli oggetti hanno proprietà, ma non hanno dei veri e propri metodi: le proprietà possono corrispondere a delle funzioni, ma esse non sono specificamente riferite all'oggetto in questione e in esse non è possibile usare il riferimento **this**
- ☒ c. ...che le proprietà di un oggetto sono sue proprie, non ricondotte a una classe di appartenenza, e il meccanismo di ereditarietà è stabilito fra un oggetto e il suo prototipo, anch'esso un oggetto, e non fra due classi
- ☐ d. ...che gli oggetti possono soltanto essere creati con un'espressione detta *object literal* nella quale vengono elencate, fra parentesi graffe, le proprietà dell'oggetto e il loro valore di partenza.

Domanda **12**

Completo

Punteggio max.: 2,00

Cosa sono le **Servlet** in un'applicazione server-side realizzata con Jakarta Web Profile?

- ☐ a. Sono il motore che gestisce il pattern Inversion of Control e si occupa di effettuare la dependency injection verso i Controller
- ☐ b. Sono il componente client-side delle applicazioni server-side realizzate con Jakarta Web Profile, e servono a definirne la user interface.
- ☒ c. Sono oggetti che ricevono la richiesta HTTP e hanno la responsabilità di elaborare la risposta; come tali, sono il fondamento di un'applicazione server-side
- ☐ d. Sono un elemento del middleware che si frappone fra il Web Container e l'applicazione vera e propria realizzata con Spring